Adaptive Traffic Signal Control Using Deep Reinforcement Learning

Student Name(s): Nandana Anish, Pranjana Rahangdale
Roll Number(s): 23FE10CSE00631, 23FE10CSE00535
Course: Reinforcement Learning, Final Year B.Tech (Computer Science & Engineering)

Abstract

Fixed-time traffic signals repeat a preset cycle and cannot adapt when traffic changes. This project formulates the control of a four-way intersection as a Markov decision process: every 5 s a reinforcement-learning (RL) agent observes the queue on each approach and chooses which direction gets green, and its reward is the negative waiting time of vehicles. A custom Gymnasium simulator models Poisson arrivals, saturation-flow discharge, yellow intervals and a maximum-green rule. Tabular Q-learning, Deep Q-Network (DQN), Double DQN and Dueling Double DQN agents were trained on randomised traffic (150 episodes, three seeds) and compared with fixed-time and queue-actuated controllers on three unseen scenarios. The Dueling Double DQN performed best in all of them, cutting the average waiting time per vehicle by 40–74% versus fixed-time and 5–15% versus actuated control.

1. Introduction

Background and motivation. Signalised intersections are a major source of urban traffic delay. Many signals run fixed-time plans computed offline from historical counts; when demand deviates from those counts, green time is wasted on empty approaches while queues grow elsewhere. Actuated controllers react to detectors through hand-tuned rules. RL instead learns a control policy from interaction by maximising a long-term reward, which makes it a natural fit for adaptive signal control.

Problem statement. Using only real-time queue lengths at one four-way intersection, learn a policy that chooses every 5 s which of two signal phases to serve so as to minimise the average waiting time per vehicle, and that generalises to unseen traffic patterns.

Objectives. (i) Formulate the task as an MDP in a Gymnasium environment; (ii) implement Q-learning, DQN, Double DQN and Dueling Double DQN agents; (iii) compare them with fixed-time and actuated control on unseen traffic; (iv) analyse learning behaviour and limitations.

2. Methodology

Environment. The simulator models one intersection with four single-lane approaches (N, S, E, W) and two phases (North–South or East–West green) in 1 s steps. Vehicles arrive as a

Poisson process whose rate depends on the scenario; a green lane discharges 0.5 veh/s, a typical saturation flow. A phase change costs a 3 s yellow interval and, as in real controllers, a phase ends automatically after 60 s (maximum green). An episode lasts 30 min (360 decisions).

State and action spaces. The state $s_t \in [0, 1]^7$ contains the four queue lengths divided by 40 (clipped), a one-hot code of the current phase and the time in that phase divided by 60 s. The action $a_t \in \{0, 1\}$ is the phase shown for the next 5 s; switching starts the yellow interval.

Reward function. The reward is minus the vehicle-seconds of waiting in the interval, scaled by 1/100 (q_i is the queue on approach i), so the episode return is proportional to the total delay:

$$r_t = -\frac{1}{100}\sum_{k=1}^{5}\sum_{i\in\{N,S,E,W\}} q_i(t+k) \tag{1}$$

RL algorithms. Tabular Q-learning [1] applies the update

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right] \tag{2}$$

over a compact 288-cell state (binned queue per direction, phase and time in phase). DQN [2] approximates $Q(s, a; \theta)$ with a network of two 128-unit ReLU layers, trained on replay-buffer minibatches against a target network θ^- copied every 1,000 steps. Double DQN [3] lets the online network choose, and the target network evaluate, the next action to reduce over-estimation:

$$y = r + \gamma\, Q\left(s', \operatorname*{argmax}_{a'} Q(s',a';\theta)\,;\ \theta^-\right) \tag{3}$$

The dueling network [4] computes $Q(s,a) = V(s) + A(s,a) - \text{mean}_{a'} A(s,a')$; the Dueling DDQN combines it with (3). The baselines are a fixed-time plan that switches every 30 s and an actuated controller that, after a 10 s minimum green, serves the direction with the longer queue.

Hyperparameters and training strategy. Table 1 lists the main hyperparameters. Exploration is ε-greedy, with ε decaying linearly from 1.0 to 0.05 over the first half of training (60% for Q-learning). Each agent was trained for 150 episodes with seeds 0, 1 and 2, each episode drawing a new random demand pattern (0.03–0.18 veh/s per lane) so the agent cannot memorise a timetable; the 30 min time limit is not treated as terminal.

Table 1. Hyperparameters used for training.

Parameter	Value	Parameter	Value
Discount γ	0.95	Replay buffer	50,000
Optimiser	Adam, lr 5×10^{-4}	Minibatch	64
Hidden layers	2 × 128, ReLU	Target update	1,000 steps
Loss	Huber	Q-learning α	0.1

Software framework and system architecture. Python [5], NumPy [6], PyTorch [7], Gymnasium [8], Matplotlib [9] and pygame-ce [10]; about 18 min of training on a laptop CPU (Fig. 1).

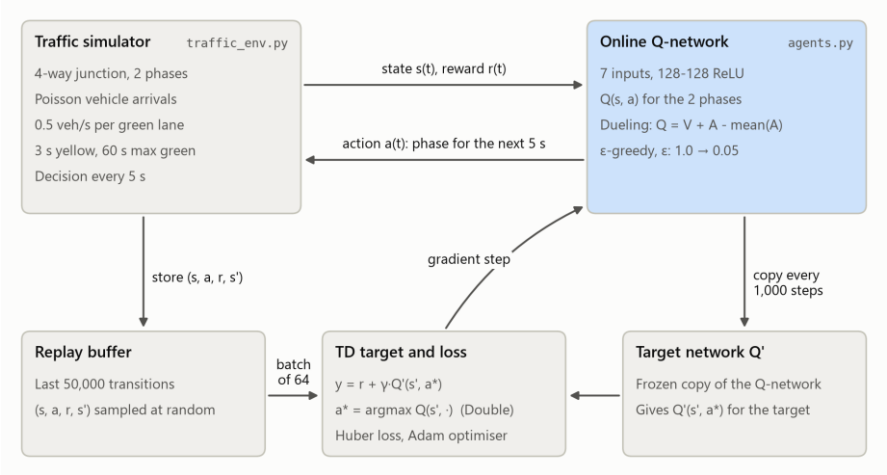


Fig. 1. System architecture and training workflow of the DQN-based signal controller.

3. Experimental Results

Evaluation protocol. Controllers ran greedily (ε = 0) on three scenarios unseen in training: uniform (0.10 veh/s per lane), asymmetric (a 0.20 veh/s main road crossing a 0.06 veh/s side street) and rush hour (successive North–South and East–West peaks), each for five episodes with fixed seeds so that all controllers faced identical arrivals.

Training curves. In Fig. 2 the agents first do worse than fixed-time control while ε is high; the DQN variants then stay below the fixed-time curve from episode 31 and below the actuated one from episode 60, settling near 8 s per vehicle, while Q-learning learns more slowly.

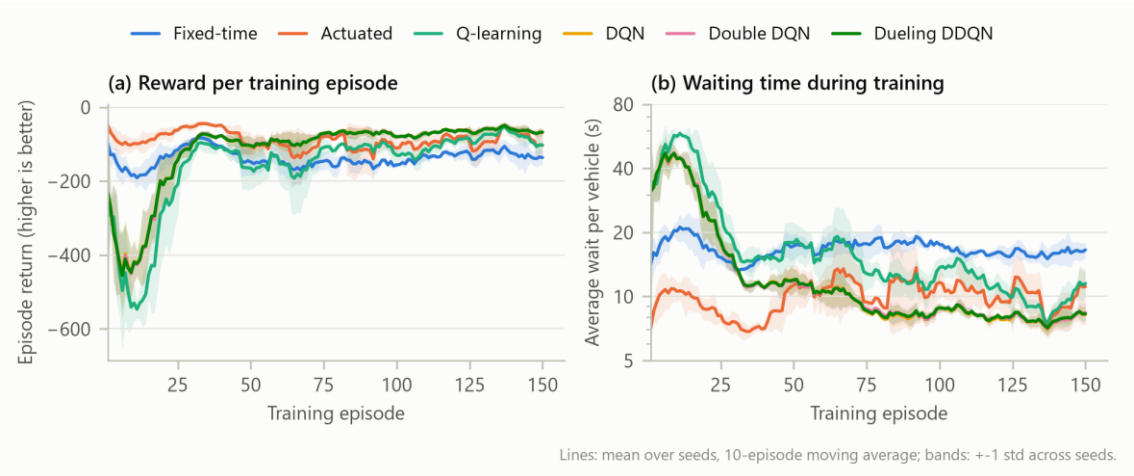


Fig. 2. Training curves: (a) episode return, (b) average waiting time (mean of 3 seeds, ±1 std band).

Comparison with baselines. The Dueling DDQN had the lowest waiting time in every scenario (Table 2): 40.2%, 74.4% and 67.3% below fixed-time and 4.7%, 15.2% and 7.9%

below actuated control. Its success rate was 87% of the 45 paired test episodes against the actuated controller and 100% against fixed-time control. Gains are largest under unbalanced demand, where a fixed 30 s/30 s split wastes green time on the quiet road.

Table 2. Test results: average wait per vehicle (s, mean ± std) and win rate against actuated control.

Controller	Uniform	Asymmetric	Rush hour	Beats actuated
Fixed-time	12.4 ± 0.4	29.0 ± 3.5	18.9 ± 3.1	0%
Actuated	7.8 ± 0.4	8.7 ± 0.7	6.7 ± 0.4	–
Q-learning	9.7 ± 1.2	11.9 ± 2.3	10.8 ± 4.3	7%
DQN	7.6 ± 0.3	7.8 ± 0.6	6.5 ± 0.7	76%
Double DQN	7.5 ± 0.3	7.5 ± 0.4	6.5 ± 0.6	69%
Dueling DDQN	**7.4 ± 0.2**	**7.4 ± 0.4**	**6.2 ± 0.4**	**87%**

Performance analysis. In the rush-hour test the fixed-time queue peaks near 20 vehicles, whereas the learned policy keeps it below 7 by extending green for the busy direction (Fig. 3). Throughput was almost identical for all controllers, so the gain is shorter waiting. Q-learning beat fixed-time control by 22–59% but was less consistent, as its coarse state merges situations that need different actions; without the maximum-green rule (which otherwise fired at most about twice per episode) one seed learned never to switch in some states. Double and Dueling DQN gave small, consistent gains over DQN, in line with [3], [4].

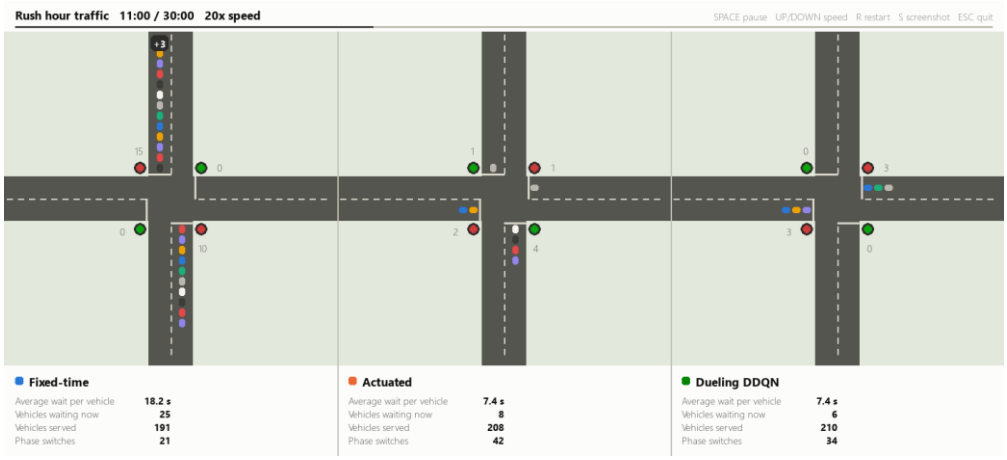


Fig. 3. Demo at minute 11 of the rush-hour test: fixed-time, actuated and Dueling DDQN.

4. Conclusion

A reinforcement-learning traffic signal controller was designed, implemented and evaluated. Using only queue lengths, the Dueling Double DQN generalised to unseen traffic and outperformed fixed-time and actuated control, while Q-learning showed the limits of tabular methods.

Limitations and future scope. The model covers one intersection with single-lane approaches, no turning traffic or pedestrians, and synthetic demand. Future work: testing in the SUMO

simulator with real counts, multi-intersection control with multi-agent RL, and policy-gradient methods such as PPO.

References

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, no. 3–4, pp. 279–292, 1992, doi: 10.1007/BF00992698.

[2] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[3] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, 2016, doi: 10.1609/aaai.v30i1.10295.

[4] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, PMLR vol. 48, New York, NY, USA, 2016, pp. 1995–2003.

[5] Python Software Foundation, "Python 3.14 documentation." [Online]. Available: https://docs.python.org/3.14/ (accessed Oct. 14, 2026).

[6] NumPy Developers, "NumPy v2.5 manual." [Online]. Available: https://numpy.org/doc/2.5/ (accessed Oct. 14, 2026).

[7] PyTorch Contributors, "PyTorch 2.14 documentation." [Online]. Available: https://docs.pytorch.org/docs/2.14/ (accessed Oct. 14, 2026).

[8] Farama Foundation, "Gymnasium documentation (v1.3.0)." [Online]. Available: https://gymnasium.farama.org/ (accessed Oct. 14, 2026).

[9] Matplotlib Development Team, "Matplotlib 3.11.2 documentation." [Online]. Available: https://matplotlib.org/3.11.2/ (accessed Oct. 14, 2026).

[10] pygame-ce Developers, "pygame-ce documentation (v2.5)." [Online]. Available: https://pyga.me/docs/ (accessed Oct. 14, 2026).